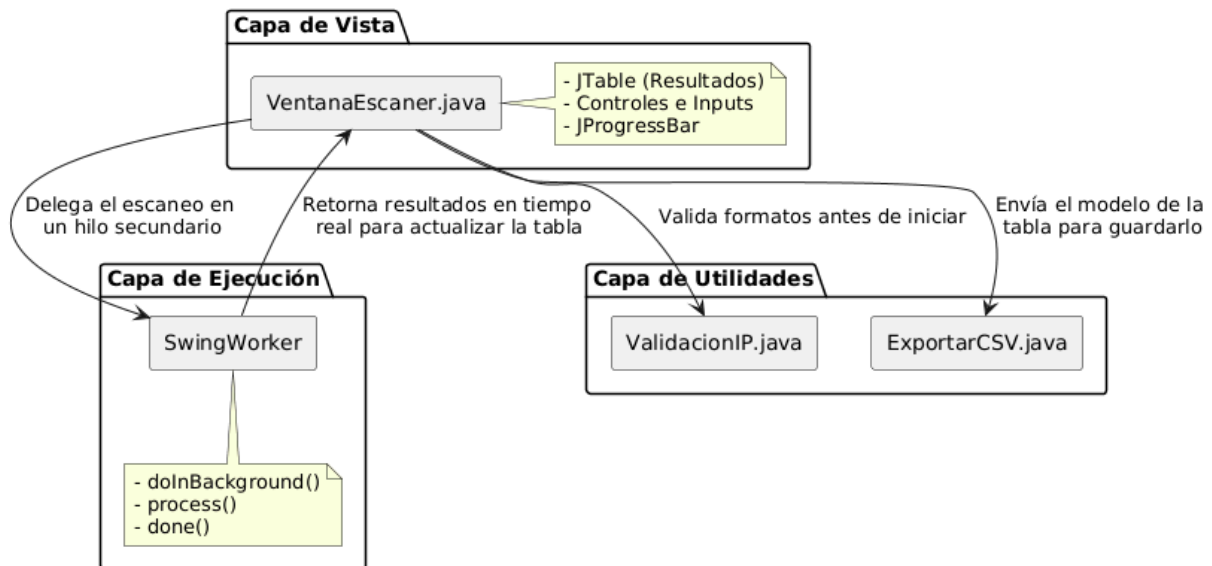


1. Propósito del Sistema

Es una herramienta para escanear una red local. Le pasás una IP de inicio y una de fin, y el programa se encarga de revisar una por una para ver qué computadoras, celulares o dispositivos están conectados. Además de decirte si están activos, te muestra cuánto tardan en responder (en milisegundos) y trata de averiguar el nombre del equipo.

2. Arquitectura del Sistema

El sistema se desarrolló utilizando un patrón de diseño basado en la separación de responsabilidades, dividiendo la interfaz gráfica de la lógica de procesamiento y las utilidades.



3. Métodos y Lógica Central

- **Hilos de fondo (SwingWorker):** Si ponía el comando ping en el mismo lugar que la pantalla, el programa se quedaba "tildado" o congelado hasta terminar de escanear. Usando esto, el escaneo corre por un lado y la pantalla sigue respondiendo por otro.
- **Comandos directos de consola (ProcessBuilder):** En vez de usar las funciones de red que trae Java (que a veces tiran errores de permisos de administrador), hice que el programa abra directamente la consola del sistema ocultamente y escriba los comandos `ping` y `nslookup`. Es más rústico pero no falla.
- **Expresiones Regulares (Regex):** Las usé para dos cosas clave: para asegurarme de que el usuario no escriba letras en lugar de IPs, y para "recortar" inteligentemente los milisegundos del texto que devuelve la consola.

- **Matemática para las IPs:** Las IPs son texto, así que no se puede hacer un ciclo "For" fácilmente. Lo que hice fue convertir esas direcciones IP a números grandes (**long**). Así es mucho más fácil ir sumando de a 1 desde la IP inicial hasta la final.

4. Elección de tecnologías

- **Java (JRE/JDK):** Seleccionado por su alta portabilidad multiplataforma. Permite que el código compilado funcione tanto en sistemas Windows como en distribuciones UNIX sin necesidad de reescribir el software.
- **Java Swing:** Se eligió esta librería gráfica porque provee componentes nativos avanzados (**JTable** con ordenamiento nativo, **JProgressBar**) sin requerir la instalación de dependencias externas.
- **Comandos OS Nativos:** La decisión de invocar **ping** y **nslookup** directamente al sistema operativo asegura que el paquete enviado sea estrictamente ICMP puro, garantizando mayor fiabilidad en el descubrimiento de equipos.

5. Problemas Encontrados y Soluciones

- **Problema 1:** Me di cuenta de que la consola de Windows y la de Linux no usan exactamente el mismo comando de ping (Windows usa **ping -n** y Linux usa **ping -c**).
 - **Solución:** Se agrego un chequeo al código para que pregunte en qué sistema operativo está corriendo y adapte el comando automáticamente.
- **Problema 2:** El programa tardaba una eternidad en escanear. Esto pasaba porque intentaba buscarle el nombre por DNS a direcciones IP que ni siquiera estaban prendidas, y se quedaba esperando.
 - **Solución:** Se puso una condición muy simple: ahora solo ejecutar el comando **nslookup** (el que busca el nombre) si el **ping** anterior funcionó.

6. Mejoras Futuras

- **Buscar puertos:** Hacer que, además de hacer ping, se fije si el equipo tiene algún servicio web (puerto 80) o carpetas compartidas abiertas.
- **Escaneo múltiple:** Ahora mismo escanea las IPs una por una en fila. Estaría bueno hacer que escanee de a 20 o 50 IPs al mismo tiempo para terminar mucho más rápido en redes gigantes.